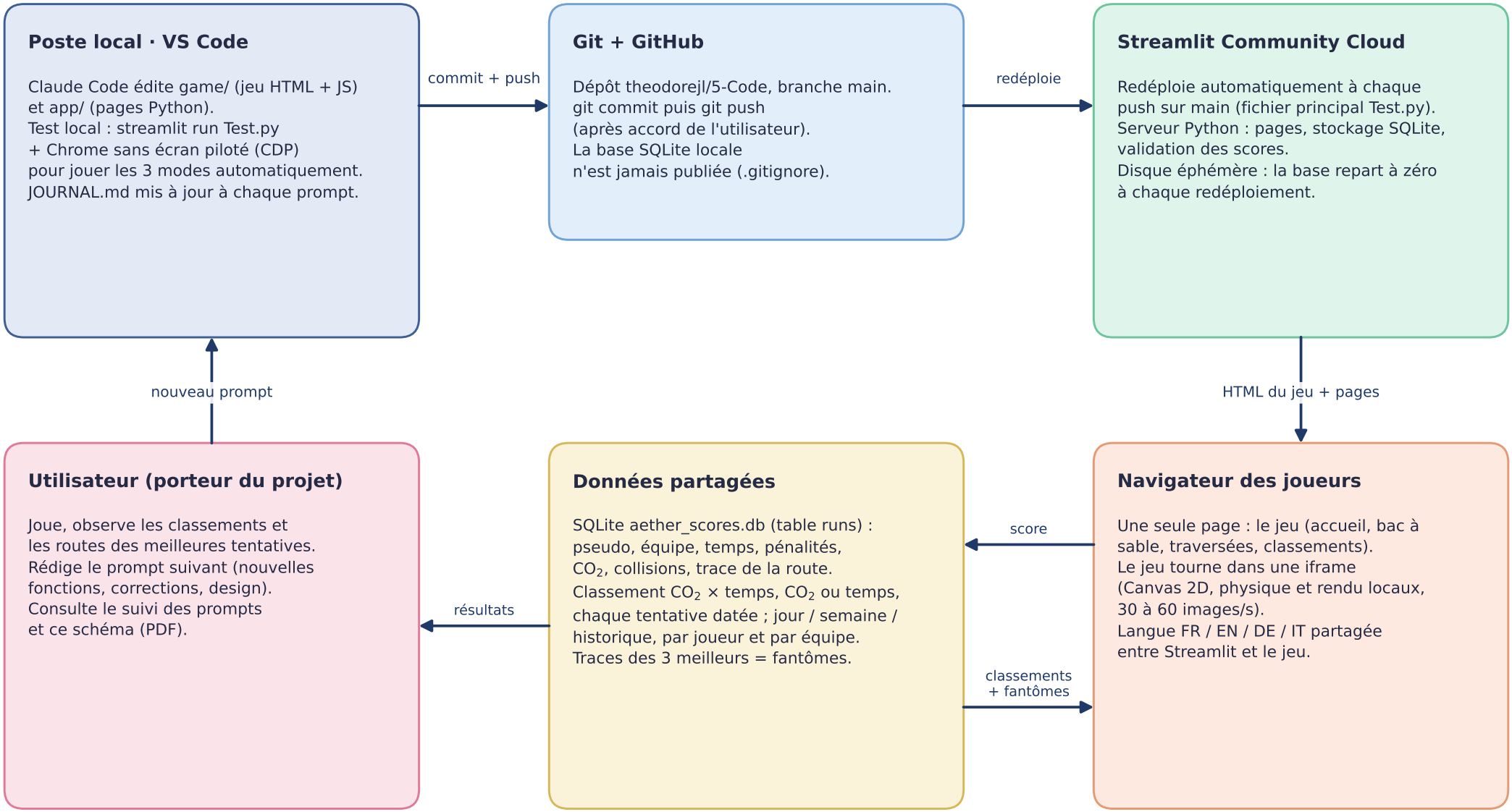
1. Boucle macro : du code au joueur

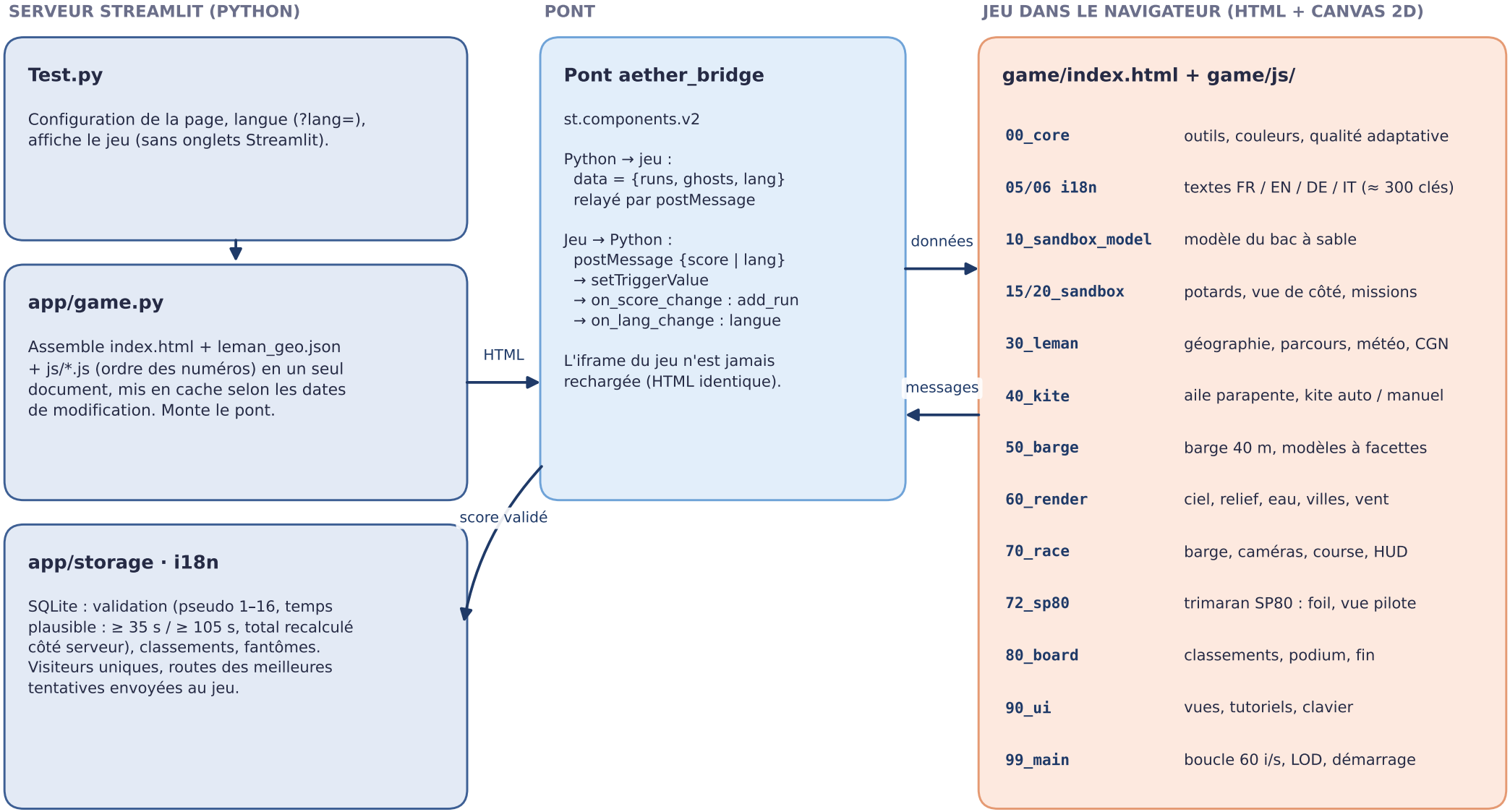
Chaque prompt suit la même boucle : modifier en local, tester, publier sur GitHub, Streamlit redéploie, les joueurs jouent.



Boucle courte (minutes) en local avec tests automatisés ; boucle longue (push) seulement après validation. Pousser sur main redéploie l'application et remet à zéro le classement en ligne.

2. Architecture logicielle

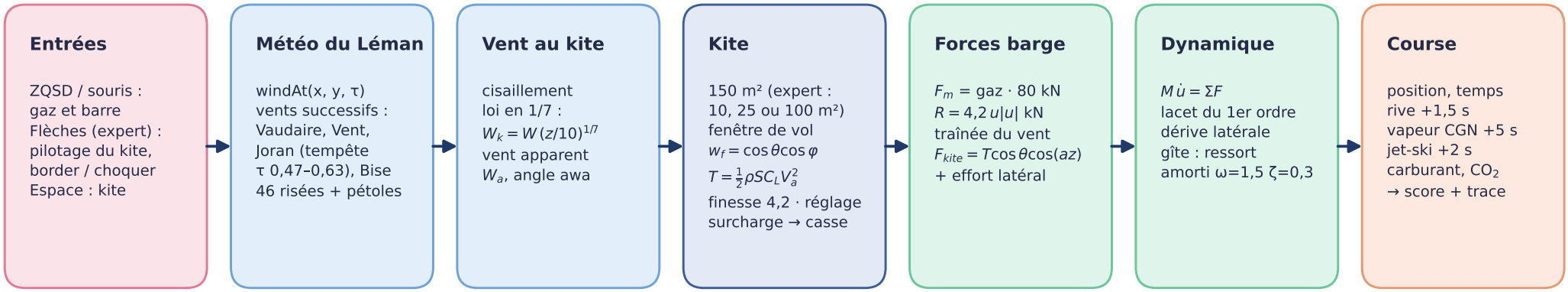
Python sert les pages et garde les scores ; tout le jeu (physique, rendu, commandes) tourne dans le navigateur.



Les modules JS partagent une seule portée (concaténés dans une fonction) : pas de doublon de code entre bac à sable, traversées et accueil (même scène 3D déplacée d'une vue à l'autre).

3. Modèles physiques (un pas de calcul par image, dt ≤ 50 ms)

Priorité : inertie de la barge chargée, traction du kite dans sa fenêtre de vol, réactions aux risées et à la tempête.



Grandeur	Débutant	Expert	Commentaire
Durée nominale (à 9,5 nd)	≈ 60 s	≈ 180 s	temps compressé : 70,7 km réels
Masse effective ressentie	80 t	190 t	barge à gravier chargée, inertie
Pilotage du kite	automatique (8)	manuel (flèches)	fenêtre de vol, choquer / border
Kite · lignes	150 m² · 200 m	10/25/100 m² · 25/100/300 m	petit = vif, grand = puissant
Tension max avant casse	95 kN	≈ 1,15 kN/m²	pénalités : +4 s / +3 s / +2 s
Taux de lacet max	17 °/s	9,5 °/s	la barre n'agit qu'avec de la vitesse
Moteur · résistance	500 kW · 80 kN	idem	R = 4,2 kN/(m/s)² · u²
Carburant · CO_2	0,215 kg/kWh	3,17 kg/kg	CO_2 évité = poussée utile du kite
Mode SP80 (1 t, kite 25 m²)	≈ 90 s	jusqu'à ~75 nd	sans moteur, foil vers 25 nd, classé au temps

Échelle 1 km réel = 60 m de jeu (1:16,7, relief ×1,3) : les angles du paysage restent réalistes ; barge, bateaux CGN et jet-skis gardent leur taille réelle.

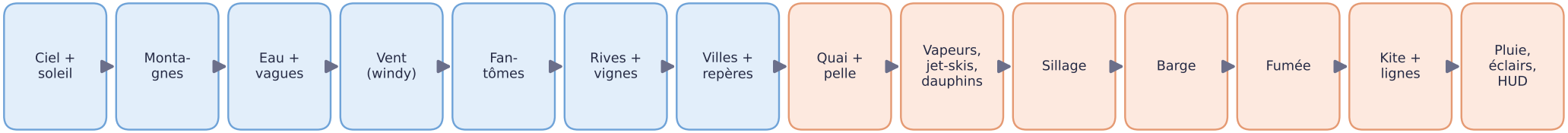
Bac à sable Puissance ∝ vitesse³ : ralentir de 8 à 7 nœuds économise plus que le kite seul ; environ 2/3 des gains viennent du ralentissement.

Même scénario météo pour tous (graine fixe) : risées, tempête, vapeurs CGN et jet-skis dépendent seulement de τ = t / durée nominale.

4. Modèles visuels, performances et outils

Rendu 3D maison sur Canvas 2D : projection équirectangulaire, ordre du peintre, faces orientées et éclairage du soleil.

ORDRE DE RENDU À CHAQUE IMAGE (DU PLUS LOIN AU PLUS PROCHE)



Caméras

Poursuite : ressort amorti derrière la barge, suit le cap en douceur.
Cabine (V) : vue depuis la timonerie.
Intro : travelling du lac vers le quai (3-2-1) avec la pelle qui charge.
Arrivée : orbite au pied du Jet d'eau.

Lumière et décor

Soleil du lever (est, 9°) au coucher sur Genève (ouest), alpenglow sur la neige.
Chablais, Dents du Midi, Évian à gauche ; Montreux, Lavaux, Lausanne, Genève à droite (aucun nom écrit).
Peu de vagues sauf en tempête.

Performances (30–60 i/s)

Qualité adaptative : si < 30 i/s pendant 2,5 s → niveau inférieur ; si > 55 i/s pendant 10 s → niveau supérieur.
Niveau 2 / 1 / 0 : résolution ×1,5/1,25/1, rangées d'eau 30/22/16, traits de vent 260/180/110, pluie 160/110/70.
Objets lointains ignorés (distance).

Outils et liens

VS Code + Claude Code (édition, tests) · Git / GitHub (dépôt theodorejl/5-Code) · Streamlit 1.65 (st.components.v2, st.iframe)
Python : SQLite (scores, visiteurs), matplotlib (ce PDF) · Navigateur : HTML, CSS, JavaScript, Canvas 2D, postMessage
Tests : Chrome sans écran + protocole DevTools (CDP) : parcours automatisés des 3 modes, 4 langues, 3 gréements et classement ; captures d'écran vérifiées.
Inspirations : bruno-simon.com (physique ludique), ig.ft.com/climate-game (enjeu climat), windy.app (visualisation du vent).